

Отчёт по лабораторной работе №11

по курсу: Архитектура компьютера и операционные системы

Иванова Анастасия Сергеевна

Содержание

1	Цель работы	5
2	Выполнение лабораторной работы	6
2.1	Задание 1. Основные команды emacs	6
2.2	Задание 2. Редактирование текста	8
2.3	Задание 3. Перемещение курсора	12
2.4	Задание 4. Управление буферами	15
2.5	Задание 5. Управление окнами	17
2.6	Задание 6. Поиск и замена	18
3	Контрольные вопросы	22
4	Вывод	25

Список иллюстраций

2.1	Запуск emacs	6
2.2	Создание файла	7
2.3	Ввод текста	7
2.4	Сохранение файла	8
2.5	Вырезание строки	9
2.6	Вставка строки	9
2.7	Выделение текста	10
2.8	Копирование области	10
2.9	Вставка области	11
2.10	Вырезание области	11
2.11	Отмена действия	12
2.12	Начало строки	13
2.13	Конец строки	13
2.14	Начало буфера	14
2.15	Конец буфера	14
2.16	Список буферов	15
2.17	Перемещение в другое окно	15
2.18	Заккрытие окна	16
2.19	Переключение буферов	16
2.20	Разделение окон	17
2.21	Работа в четырёх окнах	18
2.22	Поиск текста	19
2.23	Переключение результатов поиска	20
2.24	Выход из поиска	20
2.25	Поиск и замена	21

Список таблиц

1 Цель работы

Познакомиться с операционной системой Linux. Получить практические навыки работы с редактором Emacs.

2 Выполнение лабораторной работы

2.1 Задание 1. Основные команды emacs

1. Откроем emacs (рис. 2.1):

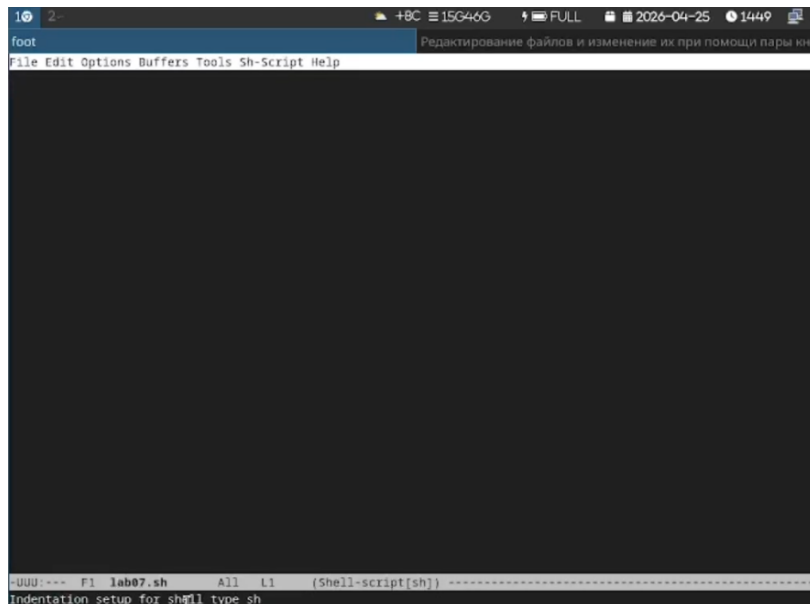


Рисунок 2.1: Запуск emacs

2. Создадим файл lab07.sh с помощью комбинации C-x C-f (рис. 2.2):

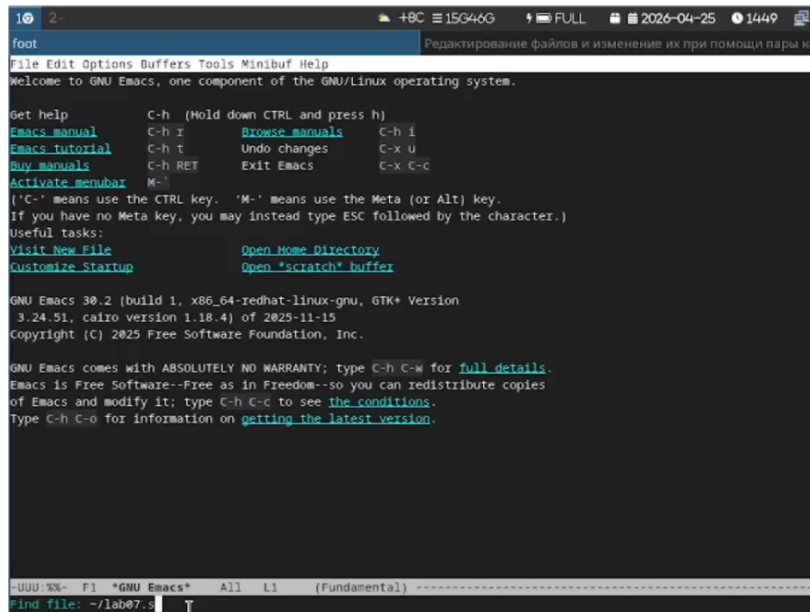


Рисунок 2.2: Создание файла

3. Наберём текст (рис. 2.3):

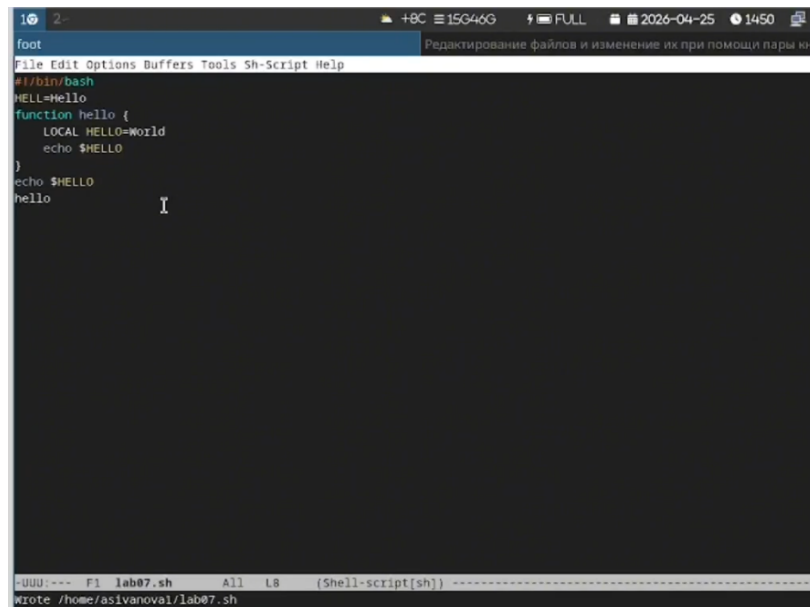


Рисунок 2.3: Ввод текста

Текст файла:

```
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

4. Сохраним файл с помощью комбинации C-x C-s (рис. 2.4):

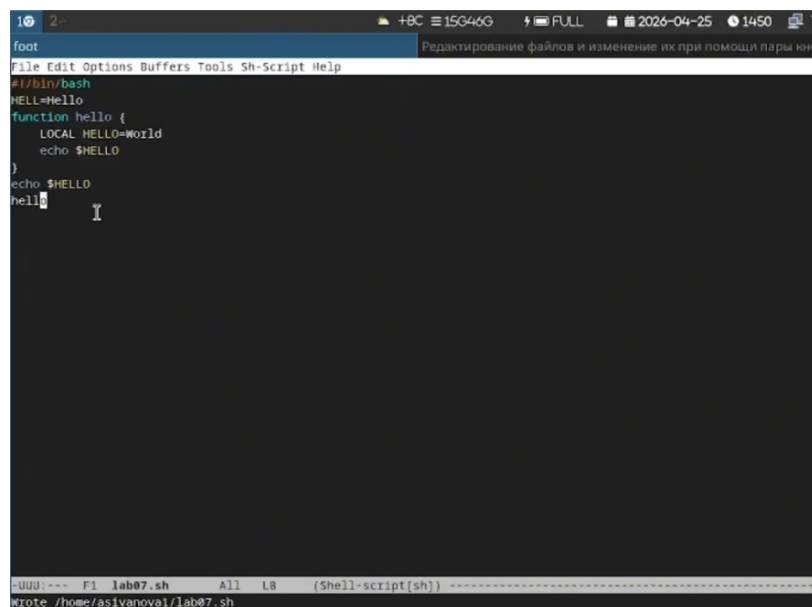
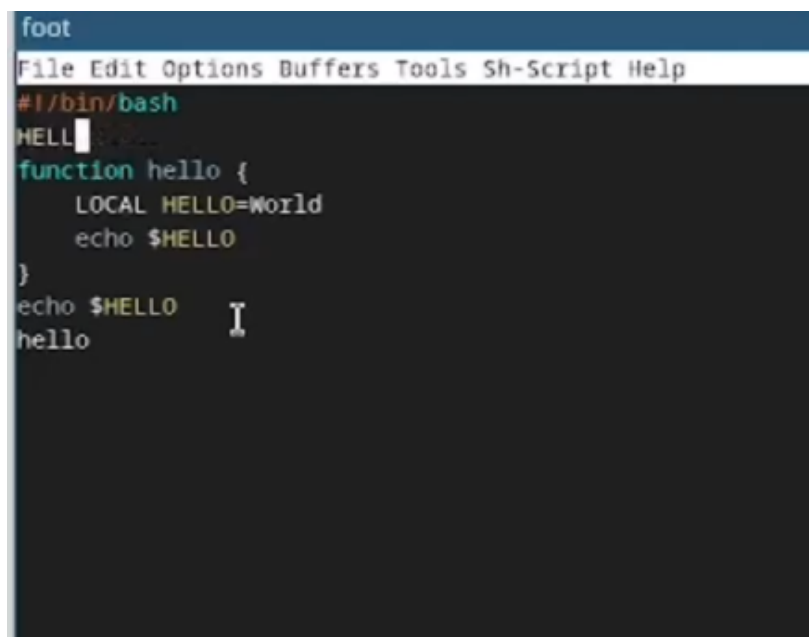


Рисунок 2.4: Сохранение файла

2.2 Задание 2. Редактирование текста

5. Проделаем с текстом стандартные процедуры редактирования:

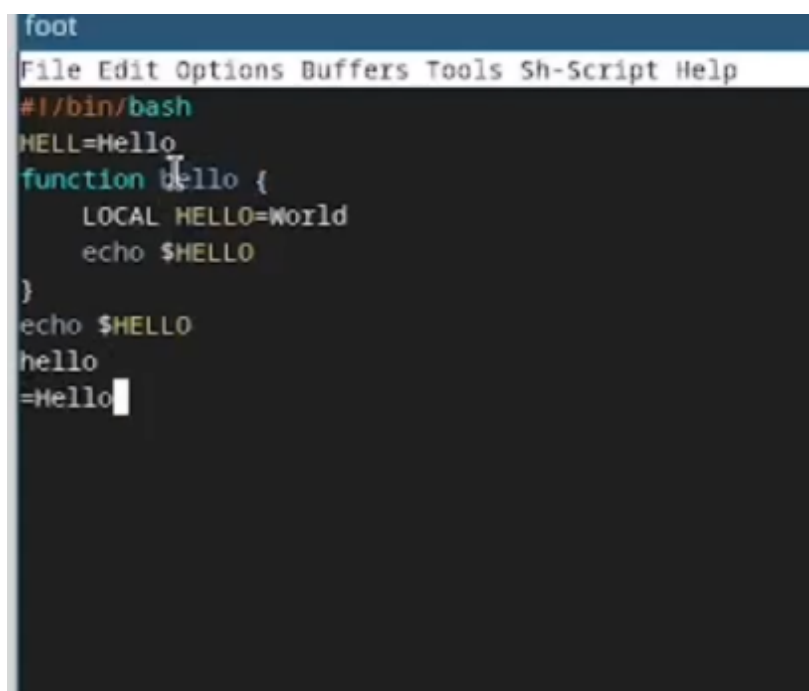
5.1. Вырежем одной командой целую строку (C-k) (рис. 2.5):



```
foot
File Edit Options Buffers Tools Sh-Script Help
#!/bin/bash
HELL=World
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

Рисунок 2.5: Вырезание строки

5.2. Вставим эту строку в конец файла (C-y) (рис. 2.6):



```
foot
File Edit Options Buffers Tools Sh-Script Help
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
HELLO=Hello
```

Рисунок 2.6: Вставка строки

5.3. Выделим область текста (C-space) (рис. 2.7):

A screenshot of a terminal window with a menu bar at the top containing 'File Edit Options Buffers Tools Sn-Script H'. The terminal has a blue background with white text. The text is a Bash script:

```
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

 A white selection cursor, resembling a vertical bar with a horizontal bar at the top, is positioned over the word 'hello' on the line 'function hello {'.

```
File Edit Options Buffers Tools Sn-Script H
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

Рисунок 2.7: Выделение текста

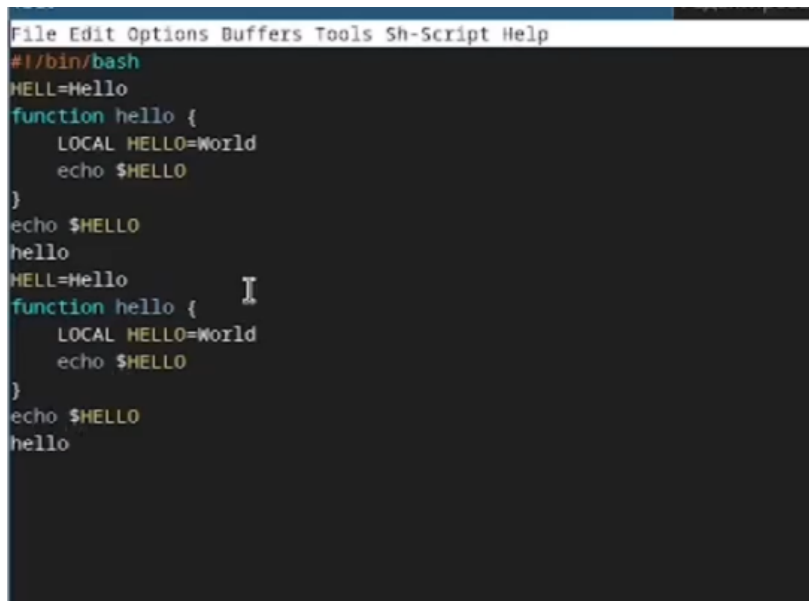
5.4. Скопируем область в буфер обмена (M-w) (рис. 2.8):

A screenshot of a terminal window, identical to the one in Figure 2.7. It shows the same Bash script on a blue background. The white selection cursor is positioned over the word 'hello' on the line 'function hello {'.

```
File Edit Options Buffers Tools Sn-Script H
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

Рисунок 2.8: Копирование области

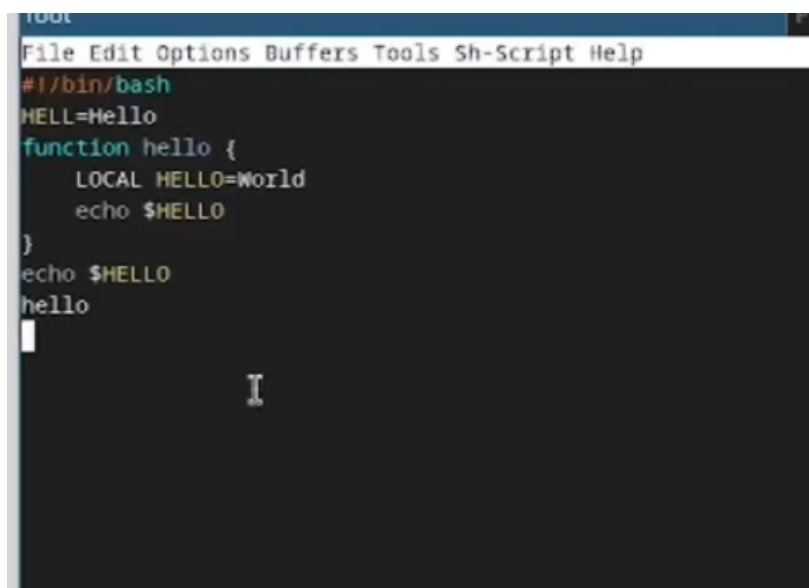
5.5. Вставим область в конец файла (C-y) (рис. 2.9):



```
File Edit Options Buffers Tools Sh-Script Help
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

Рисунок 2.9: Вставка области

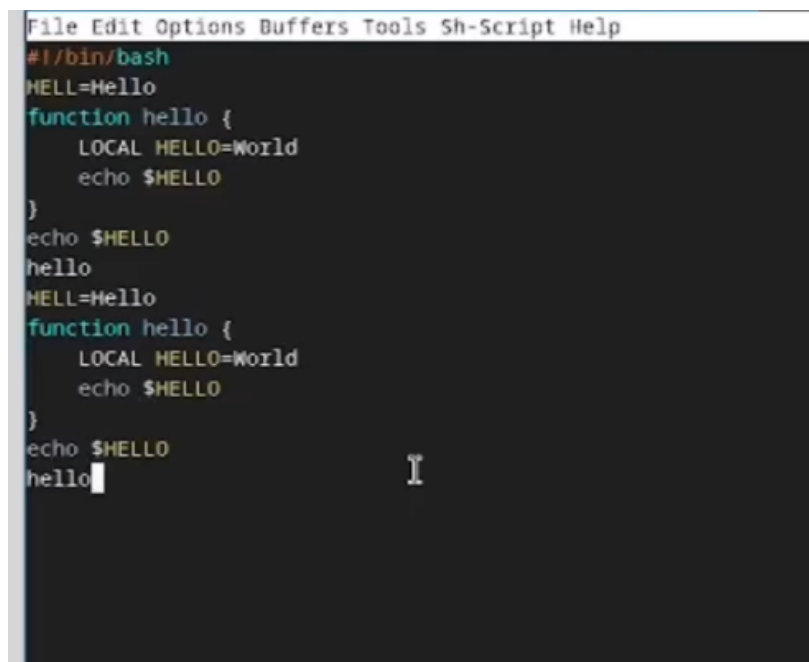
5.6. Вновь выделим эту область и вырежем её (C-w) (рис. 2.10):



```
File Edit Options Buffers Tools Sh-Script Help
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

Рисунок 2.10: Вырезание области

5.7. Отменим последнее действие (C-/) (рис. 2.11):



```
File Edit Options Buffers Tools Sh-Script Help
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

Рисунок 2.11: Отмена действия

2.3 Задание 3. Перемещение курсора

6. Научимся использовать команды по перемещению курсора:

6.1. Переместим курсор в начало строки (C-a) (рис. 2.12):

```
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

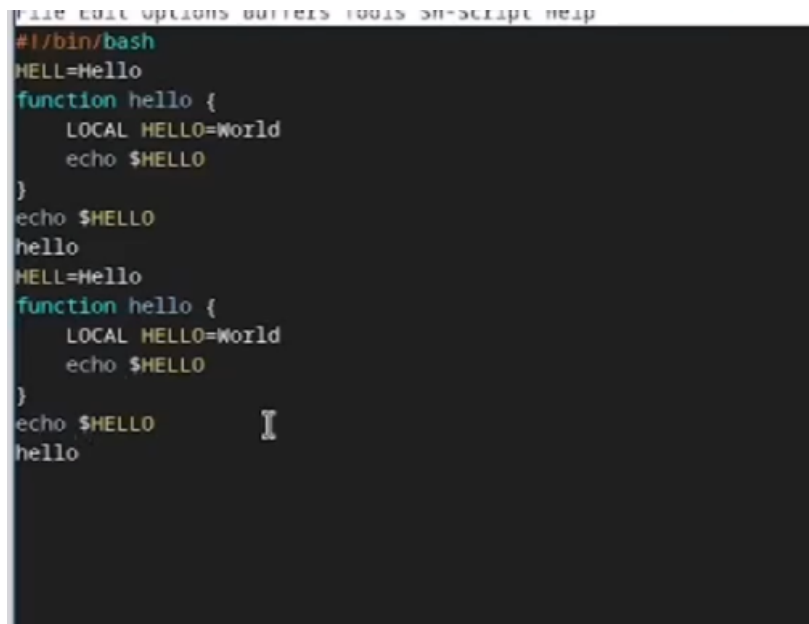
Рисунок 2.12: Начало строки

6.2. Переместим курсор в конец строки (C-e) (рис. 2.13):

```
file edit options buffers tools sh-script help
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

Рисунок 2.13: Конец строки

6.3. Переместим курсор в начало буфера (M-<) (рис. 2.14):

A screenshot of a terminal window with a dark background. The menu bar at the top includes 'File', 'Edit', 'Options', 'Buffers', 'Tools', 'Sh-Script', and 'Help'. The script content is as follows:

```
#!/bin/bash
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
HELL=Hello
function hello {
    LOCAL HELLO=World
    echo $HELLO
}
echo $HELLO
hello
```

The cursor, represented by a vertical bar, is positioned at the start of the line containing 'echo \$HELLO' in the final block of the script.

Рисунок 2.14: Начало буфера

6.4. Переместим курсор в конец буфера (M->) (рис. 2.15):

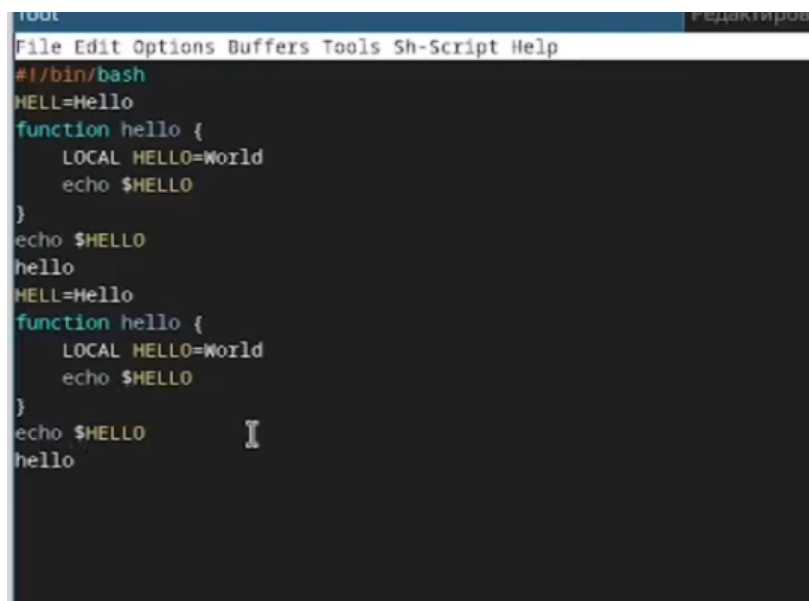
A screenshot of a terminal window, identical to the one in Figure 2.14, showing the same shell script. However, the cursor (vertical bar) is now positioned at the end of the line containing 'echo \$HELLO' in the final block of the script, after the space following the variable name.

Рисунок 2.15: Конец буфера

2.4 Задание 4. Управление буферами

7. Управление буферами:

7.1. Выведем список активных буферов на экран (C-x C-b) (рис. 2.16):

```
#!/bin/bash
HELL=Hello
function hello {
  LOCAL HELLO=World
  echo $HELLO
}
echo $HELLO
hello
HELL=Hello
function hello {
  LOCAL HELLO=World
  echo $HELLO
}
echo $HELLO
hell

-UUU:~*~ F1 lab07.sh All L15 {Shell-script[sh]} -----
CRM Buffer Size Mode File
. * lab07.sh 182 Shell-script[sh] ~/lab07.sh
% *GNU Emacs* 948 Fundamental
% *scratch* 147 Lisp Interaction
%* *Messages* 2274 Messages
%* *Completions* 1064 Completion List
```

Рисунок 2.16: Список буферов

7.2. Переместимся во вновь открытое окно со списком буферов (C-x o) (рис. 2.17):

```
File Edit Options Buffers Tools Buffer Menu Help
#!/bin/bash
HELL=Hello
function hello {
  LOCAL HELLO=World
  echo $HELLO
}
echo $HELLO
hello
HELL=Hello
function hello {
  LOCAL HELLO=World
  echo $HELLO
}
echo $HELLO
hell

-UUU:~*~ F1 lab07.sh All L15 {Shell-script[sh]} -----
CRM Buffer Size Mode File
. * lab07.sh 182 Shell-script[sh] ~/lab07.sh
% *GNU Emacs* 948 Fundamental
% *scratch* 147 Lisp Interaction
%* *Messages* 2274 Messages
%* *Completions* 1064 Completion List

I

-UUU:~*~ F1 *Buffer List* All L1 {Buffer Menu} -----
```

Рисунок 2.17: Перемещение в другое окно

7.3. Закроем это окно (C-x 0) (рис. 2.18):

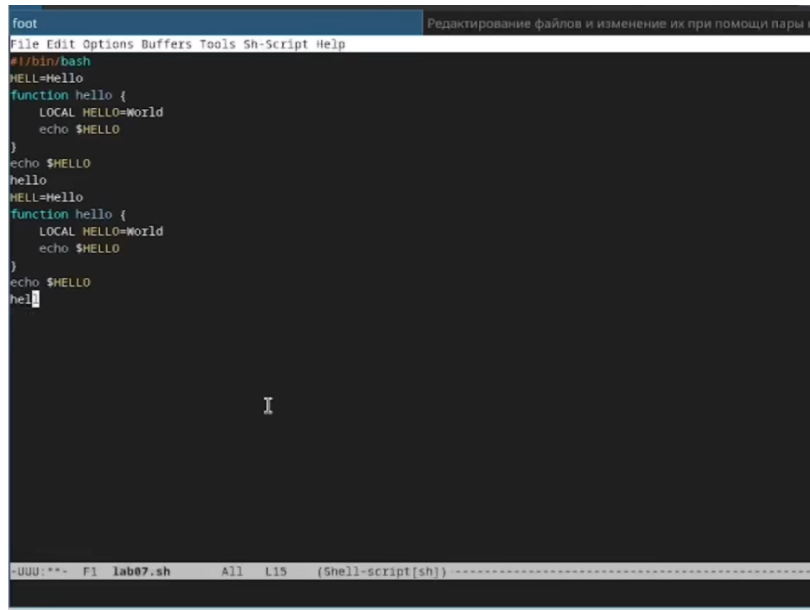


Рисунок 2.18: Заккрытие окна

7.4. Переключимся между буферами без вывода списка на экран (C-x b) (рис. 2.19):

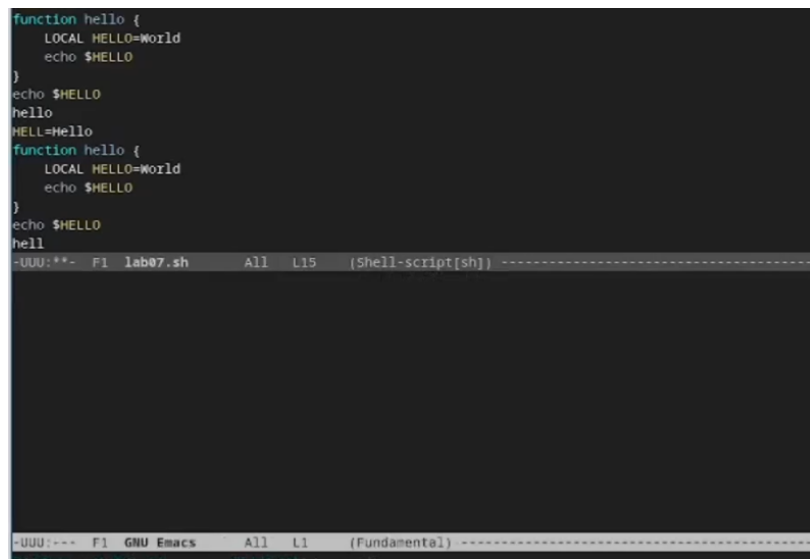


Рисунок 2.19: Переключение буферов

2.5 Задание 5. Управление окнами

8. Управление окнами:

8.1. Разделим фрейм на 4 части: сначала разделим по вертикали (С-х 3), затем каждое окно по горизонтали (С-х 2) (рис. 2.20):

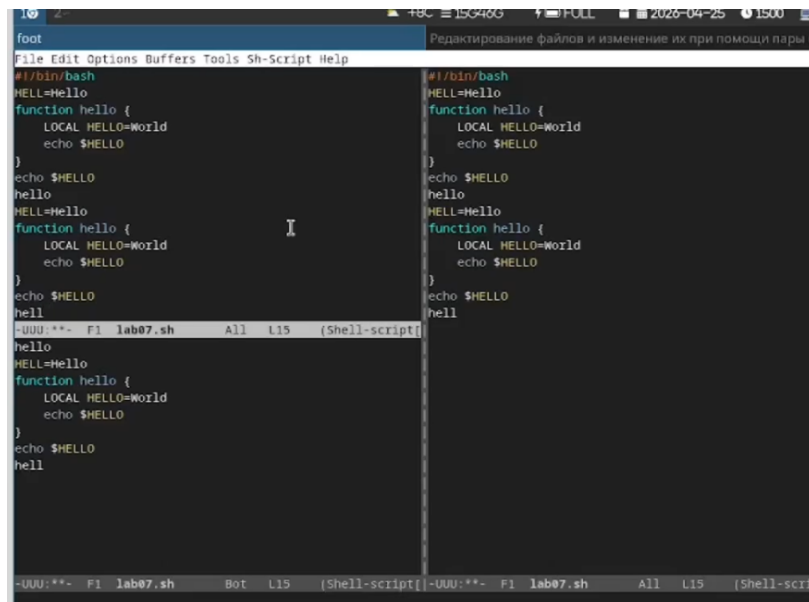


Рисунок 2.20: Разделение окон

8.2. В каждом из четырёх созданных окон откроем новый буфер (файл) и введём несколько строк текста (рис. 2.21):

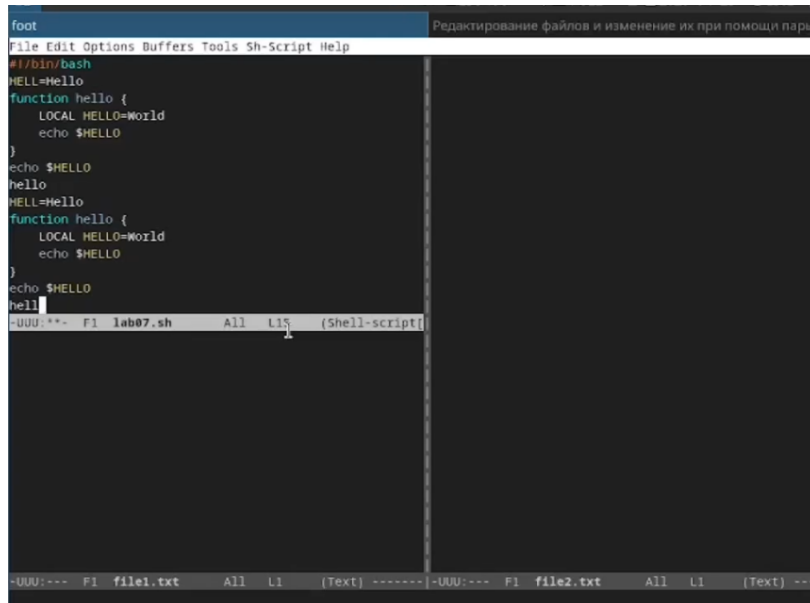


Рисунок 2.21: Работа в четырёх окнах

2.6 Задание 6. Поиск и замена

9. Режим поиска:

9.1. Переключимся в режим поиска (C-s) и найдём несколько слов, присутствующих в тексте (рис. 2.22):

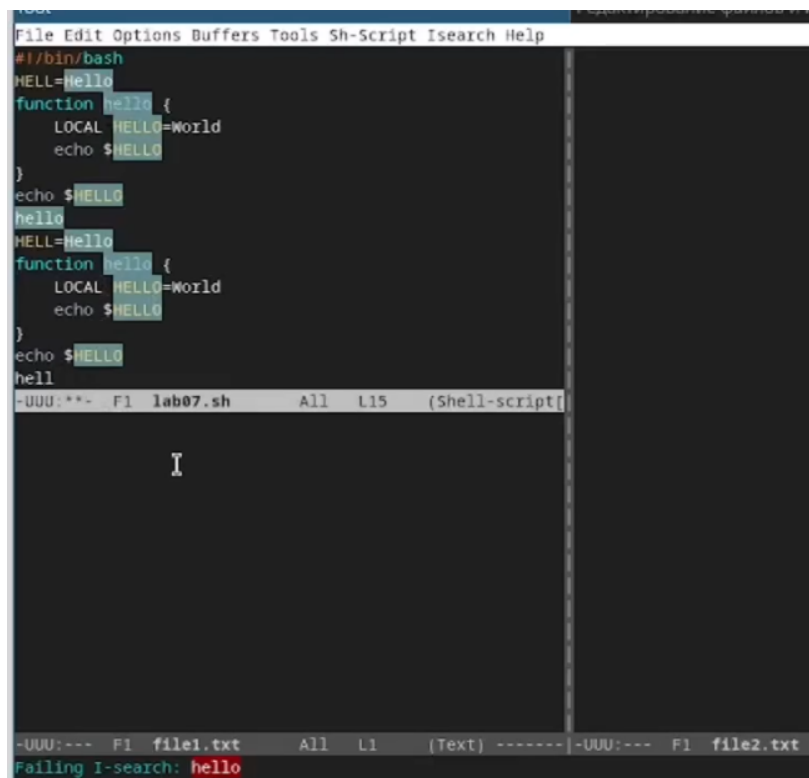


Рисунок 2.22: Поиск текста

9.2. Переключимся между результатами поиска, нажимая C-s (рис. 2.23):

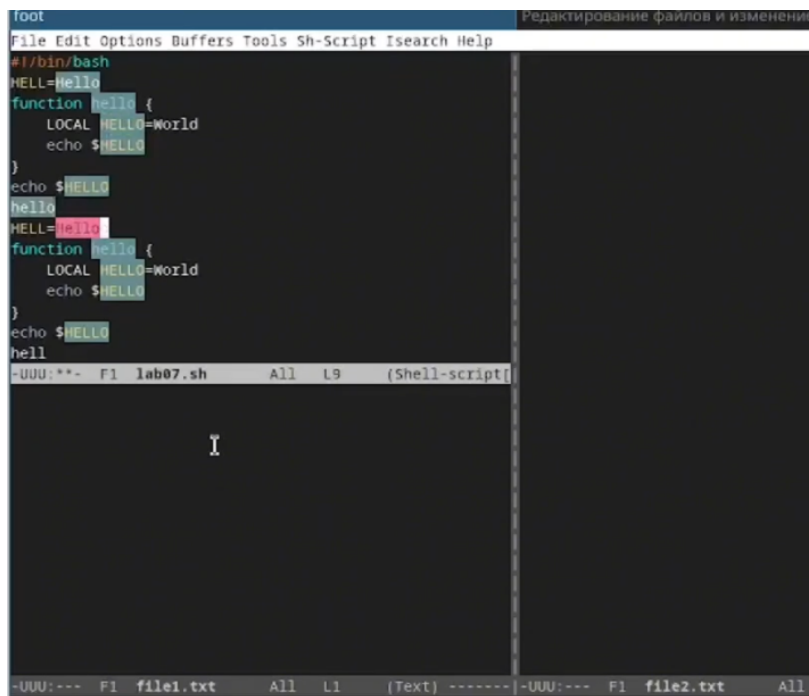


Рисунок 2.23: Переключение результатов поиска

9.3. Выйдем из режима поиска, нажав C-g (рис. 2.24):

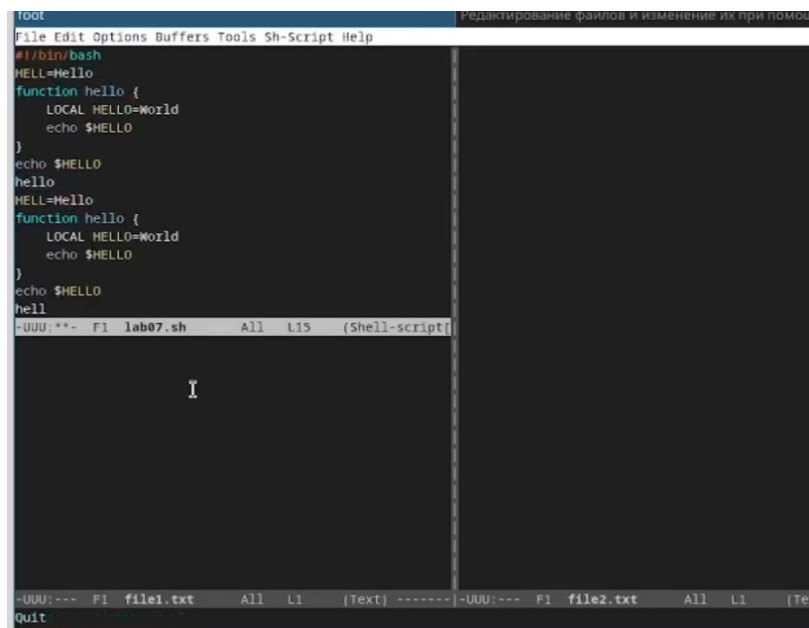


Рисунок 2.24: Выход из поиска

9.4. Перейдём в режим поиска и замены (M-%), введём текст, который следу-

ет найти и заменить, затем текст для замены. После подсветки результатов нажмём ! для подтверждения замены (рис. 2.25):

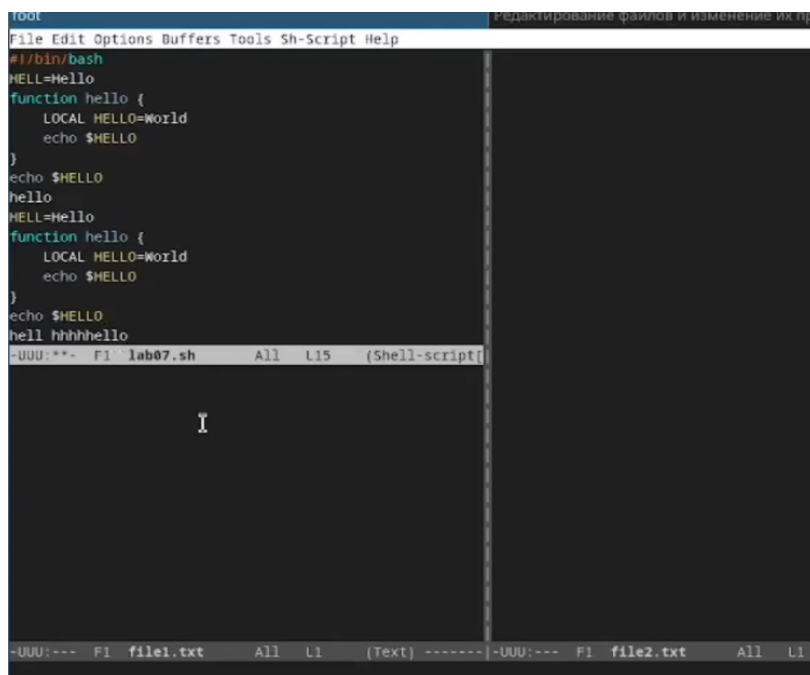


Рисунок 2.25: Поиск и замена

3 Контрольные вопросы

1. Кратко охарактеризуйте редактор emacs.

Emacs — мощный экранный редактор текста, написанный на языке высокого уровня Elisp. Поддерживает множество режимов, расширений и может быть настроен под любые задачи.

2. Какие особенности данного редактора могут сделать его сложным для освоения новичком?

Сложность в большом количестве комбинаций клавиш, неочевидных для новичка. Многие действия выполняются через Ctrl и Meta (Alt), а не через меню. Также Emacs имеет собственную философию и терминологию (буферы, фреймы, окна).

3. Своими словами опишите, что такое буфер и окно в терминологии emacs'а.

- **Буфер** — объект, содержащий какой-либо текст (файл, результаты компиляции, сообщения, справку). Буфер может быть не связан с файлом на диске.
- **Окно** — прямоугольная область фрейма, отображающая один буфер. Один фрейм может содержать несколько окон.

4. Можно ли открыть больше 10 буферов в одном окне?

Да, количество буферов не ограничено. Окно показывает один буфер в каждый момент времени, но переключаться между буферами можно в любом количестве.

5. Какие буферы создаются по умолчанию при запуске emacs?

По умолчанию создаются: `*scratch*` (черновик для временных записей), `*Messages*` (буфер с системными сообщениями), `*GNU Emacs*` (приветственное окно).

6. Какие клавиши вы нажмёте, чтобы ввести следующую комбинацию C-c | и C-c C-|?

- C-c | — Ctrl + c, затем | (Shift +)
- C-c C-| — Ctrl + c, затем Ctrl + Shift + \

7. Как поделить текущее окно на две части?

- По горизонтали (верх/низ): C-x 2
- По вертикали (лево/право): C-x 3

8. В каком файле хранятся настройки редактора emacs?

Настройки хранятся в файле `~/.emacs` или в каталоге `~/.emacs.d/init.el`.

9. Какую функцию выполняет клавиша ← и можно ли её переназначить?

Клавиша ← (влево) перемещает курсор на один символ влево. Да, можно переназначить через `global-set-key` в конфигурационном файле.

10. Какой редактор вам показался удобнее в работе vi или emacs? Поясните почему.

Emacs удобнее для повседневной работы благодаря графическому интерфейсу, меню и большому количеству расширений. vi удобнее при работе в консоли и на удалённых серверах, так как он есть в любой Unix-системе по умолчанию.

4 Вывод

Мы познакомились с операционной системой Linux и получили практические навыки работы с редактором Emacs.